Understanding CNN Generalisation with Data Augmentation on CIFAR-10

A TensorFlow CNN case study using CIFAR-10

Student ID: 24139736

Dev Article:
https://dev.to/maxwell_ororho/understanding-cnn-generalisation-with-data-augmentation-tensorflow-cifar-10-5o7

GitHub repository:
https://github.com/maxwellororho/Understanding-CNN-Generalisation-with-Data-Augmentation-on-CIFAR-10

Video tutorial:
https://drive.google.com/file/d/1XAd5qsRyVIIibQYTeHygF6306TvsOROR/view?usp=sharing

1. Introduction

If you have ever trained a neural network and watched it score 95% on the training set but only 70% on the test set, you have already met overfitting. It is one of the most common problems in deep learning, and it was the first thing I kept running into when I started experimenting with image classification. The network was essentially memorising the training images rather than learning anything useful.

One of the most practical fixes I came across is data augmentation. The idea is refreshingly simple: instead of showing the model the same images over and over, you randomly flip, rotate, or shift them on the fly so the model sees a slightly different version each time. It never gets to memorise any single image because that image never looks exactly the same twice [2].

In this tutorial, I want to explore a question that I found surprisingly hard to get a straight answer to: how much augmentation is actually useful? I ran three experiments on the CIFAR-10 dataset using the same CNN architecture, changing only the augmentation strategy: no augmentation, light augmentation, and strong augmentation. By the end, I hope you will have a clearer sense of when augmentation helps, when it gets in the way, and how to make that call in your own projects.

2. Background Theory

2.1 Convolutional Neural Networks

A CNN is built to recognise spatial patterns in images. It works by passing the image through a series of convolutional layers, each of which applies a set of learned filters to detect things like edges, textures, and shapes. After each convolutional block, a pooling layer shrinks the spatial dimensions and makes the features a bit more robust to small shifts in position. Eventually, the extracted feature maps are flattened and fed into dense layers that output a class prediction.

In my model, I also added Batch Normalisation after each convolutional layer. This normalises the activations during training, which tends to stabilise learning and also provides a small amount of regularisation as a bonus.

2.2 Underfitting and Overfitting

Before jumping into augmentation, it helps to be clear about what we are actually trying to fix. There are three situations you can find yourself in when training a model:

Concept	Description
Underfitting	The model is too simple or has not trained long enough. It struggles to learn the data and performs poorly on both training and validation sets.
Overfitting	The model has learned the training data too well, including its noise. Training accuracy is high but validation accuracy lags behind.
Good generalisation	The sweet spot. The model has learned the right patterns and performs consistently on both training and unseen data.

Overfitting is the one we are trying to tackle here. The telltale sign is a growing gap between training and validation curves as training progresses. The training loss keeps dropping while the validation loss stops improving or even starts going back up. Once you know what that looks like, you will spot it everywhere.

2.3 Data Augmentation

Data augmentation is my favourite regularisation trick because it costs almost nothing to add and it makes intuitive sense. By randomly transforming training images at each epoch, you force the model to learn features that stay consistent across different positions, angles, and scales rather than memorising exact pixel arrangements [2].

The key operations I used in this tutorial are:

- Horizontal flipping: mirrors the image left-right, which works well for objects that do not have a fixed orientation
- Random rotation: tilts the image by a small angle, simulating a camera that is not perfectly level
- Width and height shifts: slides the object around within the frame, so the model cannot rely on the subject always being centred
- Zoom: zooms in or out slightly, helping the model handle objects at different scales

There is a catch though. If you crank up the transformations too aggressively, especially on small low-resolution images, you can end up with training samples that are distorted beyond recognition. At that point you are not helping the model generalise, you are just making the training data harder and noisier. This is the trade-off I wanted to investigate [3].

3. Dataset

I used CIFAR-10 for this tutorial [4], which is one of the most well-known benchmarks in image classification. It contains 60,000 colour images spread across ten classes, with each image being a small 32x32 pixels. That tiny resolution is actually quite important for this tutorial because it is what makes strong augmentation risky, as I will show later.

I split the data as follows:

Split	Images	Purpose
Training	40,000	Used to update the model weights during training
Validation	10,000	Monitored during training to catch overfitting early
Test	10,000	Used only once at the end for the final accuracy figure

The ten classes cover a nice range of subjects: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. All pixel values are rescaled to [0, 1] before training. One thing worth knowing going in is that some classes are genuinely tricky to tell apart at this resolution. A cat and a dog can look very similar when you only have 32x32 pixels to work with, and the same goes for automobiles and trucks. You will see this show up clearly in the confusion matrix later.

4. Methodology

4.1 CNN Architecture

To make a fair comparison between augmentation strategies, I kept the model exactly the same across all three experiments. The only thing that changes is what happens to the training images before the model sees them. The network has 357,706 total parameters (357,258 trainable, with 448 frozen in the Batch Normalisation layers).

All three models are trained with the Adam optimiser, categorical cross-entropy loss, a batch size of 64, and 15 epochs. I also kept Dropout at rate 0.5 in all three conditions so that the only variable being tested is the augmentation itself.

4.2 Augmentation Strategies

I set up three conditions. The baseline gets no augmentation at all, which gives me a reference point to measure against. The light augmentation condition uses gentle transformations that I felt were unlikely to distort a 32x32 image too badly. The strong augmentation condition pushes those parameters further and adds zoom on top. Here is the full comparison:

Transformation	Baseline	Light Augmentation	Strong Augmentation
Horizontal flip	No	Yes	Yes
Rotation range	None	10 degrees	20 degrees
Width shift	None	10%	15%
Height shift	None	10%	15%
Zoom range	None	None	20%

Figure 1 shows what these transformations actually look like applied to the same image. You can see that the light augmentation examples still look like the original subject with just

subtle differences. The strong augmentation examples are noticeably more distorted. Keep that in mind when we get to the results, because it goes a long way towards explaining why strong augmentation does not perform as well.

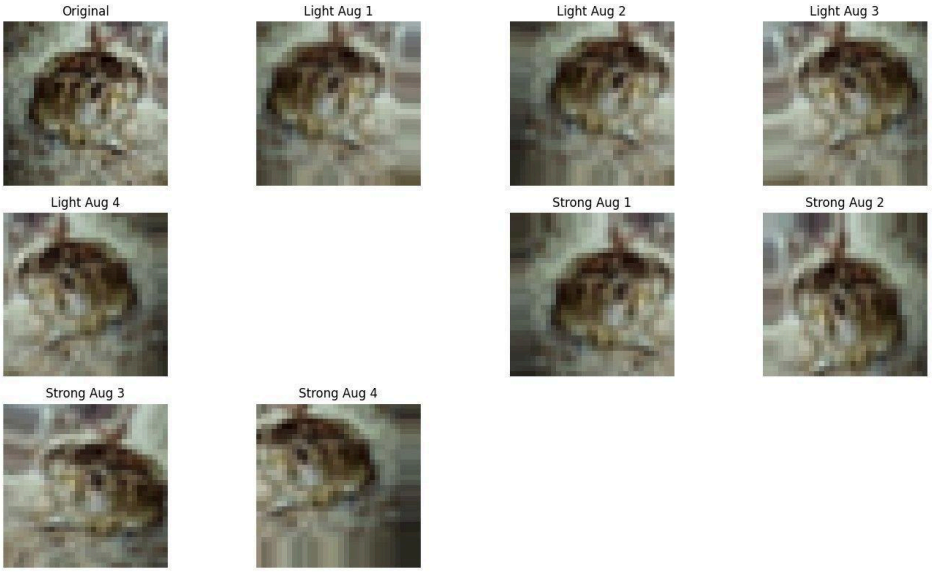

Figure 1. *The same CIFAR-10 image shown in its original form alongside four light augmentation and four strong augmentation examples. You can see light augmentation keeps the image clearly recognisable, while strong augmentation introduces more significant distortion at this resolution.*

5. Results and Analysis

5.1 Test Accuracy

After training all three models for 15 epochs, I evaluated them on the held-out test set. The results are shown below.

Model	Test accuracy
Baseline (no augmentation)	0.752
Light augmentation	0.750
Strong augmentation	0.692

Honestly, the light augmentation result surprised me a little. It matched the baseline almost exactly, coming in at 0.750 versus 0.752. That is effectively no difference. Strong augmentation, on the other hand, dropped to 0.692, which is a noticeable six percentage point fall. My interpretation is that with 40,000 training images and Dropout already in the model, there was simply not much overfitting left for augmentation to fix. And with 32x32 images, the stronger transformations ended up making the training data harder rather than more diverse.

5.2 Validation Curves

Figure 2 shows how validation accuracy and validation loss evolved over the 15 epochs for all three models. The baseline and light augmentation lines track each other really closely

throughout, which matches their near-identical final accuracy. There is some wobble in the later epochs for both, which is pretty normal at this dataset size and is nothing to worry about.

The strong augmentation line is the interesting one. It starts lower and never quite catches up. The model is learning, but more slowly, because it is trying to make sense of heavily distorted 32x32 images from the very first epoch. This is exactly the kind of curve that should make you reconsider your augmentation settings in a real project.

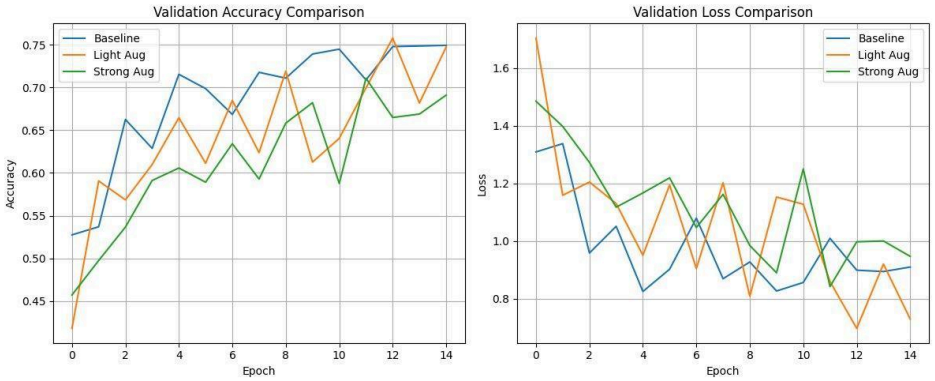

Figure 2. *Validation accuracy (left) and validation loss (right) across 15 epochs. The baseline and light augmentation models stay close to each other throughout. The strong augmentation model learns more slowly and settles at a lower final accuracy.*

5.3 Confusion Matrix Analysis

Figure 3 shows the confusion matrix for the baseline model. Each row is the true class and each column is the predicted class, so a perfect model would have all the counts sitting on the diagonal. Looking at it, the model does really well on classes like ship, frog, and automobile, which tend to have distinctive appearances even at low resolution. The trickier cases are exactly the ones you would expect: cat versus dog, and automobile versus truck. At 32x32 pixels, these pairs genuinely look very similar, and I do not think any amount of augmentation would fully close that gap. It is more of a resolution problem than a training problem.

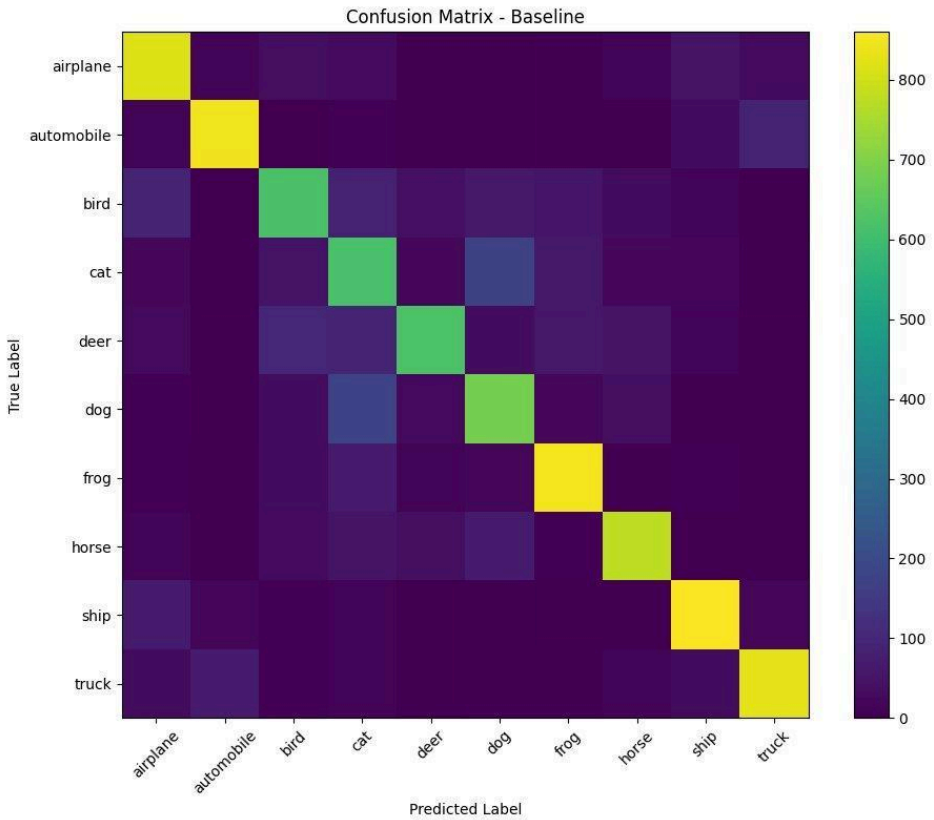

Figure 3. *Confusion matrix for the baseline model on the CIFAR-10 test set. The brightest off-diagonal cells are between cat and dog, and between automobile and truck, reflecting how similar these classes look at 32x32 resolution.*

6. Discussion

Going into this, I expected light augmentation to clearly outperform the baseline and strong augmentation to do even better. That is not what happened, and I think the reason is instructive. With 40,000 training images, Dropout, and Batch Normalisation already in the model, there was not a severe overfitting problem to solve in the first place. Augmentation is most valuable when your training set is small or when your model is large relative to the data. Here, the model was already reasonably well-regularised without it.

The strong augmentation result is a useful reminder that more is not always better. At 32x32 pixels, a 20% zoom or a 15% shift can seriously distort the subject. You are essentially introducing harder and noisier training examples, which slows down learning without improving generalisation. I found that checking what the augmented images actually look

like, as in Figure 1, is one of the best sanity checks you can do before committing to a set of augmentation parameters.

If I were to extend this work, I would be curious to try the same experiment with a much smaller training set, say 5,000 images, where I would expect augmentation to have a much more visible impact. I would also like to explore more modern techniques like CutMix or RandAugment, which have shown strong results on CIFAR-10 and may handle the resolution constraint more gracefully than simple geometric transformations [3].

7. Ethics and Accessibility

CIFAR-10 is a safe dataset to work with from a privacy perspective, as it only contains photographs of objects and animals with no identifiable people [4]. That said, it is worth being cautious about how results on CIFAR-10 are interpreted. The benchmark was collected under specific conditions, and a model that scores well here may not behave the same way when deployed on real-world images with different lighting, resolution, or camera angles. I would be careful about making strong claims based on CIFAR-10 performance alone.

For accessibility, I made sure all the figures in this tutorial use enough contrast to be readable without relying on colour alone, and every figure has a caption that describes what it shows so it makes sense even if the image cannot be displayed. The Jupyter notebook uses structured markdown headings throughout so that it works properly with screen readers, and the tutorial text is set in a 12pt serif font for comfortable reading.

8. Conclusion

This tutorial set out to answer a simple question: does more augmentation mean better generalisation? The short answer, at least in this setting, is no. With a CNN of 357,706 parameters trained on 40,000 CIFAR-10 images, the baseline achieved a test accuracy of 0.752. Light augmentation came in at 0.750, essentially the same. Strong augmentation dropped to 0.692, making it the worst of the three.

The lesson I took away is that augmentation needs to be matched to the problem. On small or low-resolution images, aggressive transformations do more harm than good. The best approach is to start conservatively, look at what your augmented images actually look like, watch your validation curves, and only add more intensity if you can see clear signs of overfitting that the current setup is not handling. That way you are solving an actual problem rather than introducing a new one.

References

[1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," in Advances in Neural Information Processing Systems (NIPS), 2012, pp. 1097-1105.

[2] C. Shorten and T. M. Khoshgoftaar, "A Survey on Image Data Augmentation for Deep Learning," Journal of Big Data, vol. 6, no. 1, 2019.

[3] L. Taylor and G. Nitschke, "Improving Deep Learning with Generic Data Augmentation," in Proc. IEEE Symposium Series on Computational Intelligence, 2018.

[4] A. Krizhevsky, "Learning Multiple Layers of Features from Tiny Images," Technical Report, University of Toronto, 2009.

[5] M. Abadi et al., "TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems," 2015. [Online]. Available: https://www.tensorflow.org

[6] F. Chollet et al., "Keras Documentation," 2015-2024. [Online]. Available: https://keras.io